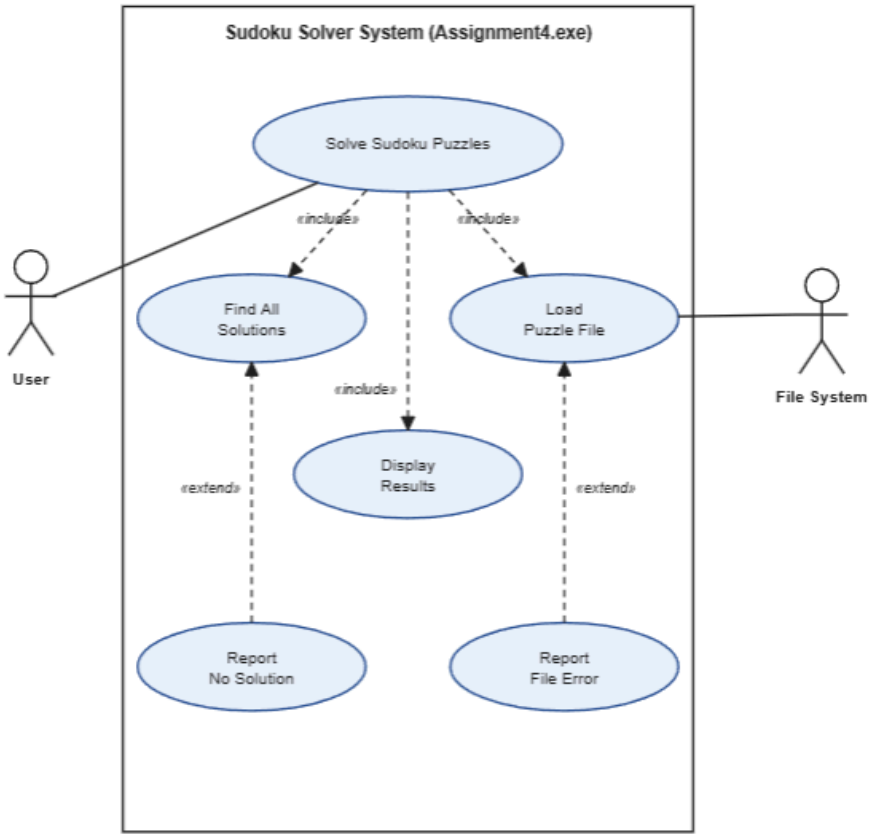
GenAI used - Claude by Anthropic.
I prompted the AI to produce a diagram based off the context of the raw commentless code - It retuned the following diagram:



Following was written by me - I was unsure if you wanted us to generate this so I wrote this to be safe.

1. The primary use case is the sudoku puzzle solver.
2. The user runs the program and it solves 5 provided puzzles. This use case takes a puzzle on an 9x9 grid and solves it using the sudoku rules, where rows, cols and the 9 non overlapping 3x3 mini grids all do not share any values, with 1-9 being the legal values. For each it uses recursive backtracking to determine all possible solutions, then it prints those solutions. Prints no solution if none are found.
3. Step by step flowchart:
    1. First the user runs the .exe
    2. The program takes the next puzzle (from 1-5)
    3. The program initializes 81 blank indexes in the grid
    4. The program then populates the blank page with the non-blank spots from the initial puzzle.
    5. The grid searches for the next free empty cell (if none go to step 8 you hit the base case)
    6. The program determines what numbers are legal for that spot.
    7. For each legal digit it places it in and calls the solver recursively if it fully resolves, it is invalid and it backtracks trying a different digit in that place.

8. Once a solution is found +1 to the solutions count, print the solution

9. One all possible solutions are exhausted, prints none found if none were found otherwise finishes running for that puzzle

10. Repeat back to 2 if there is are more puzzles otherwise finishes running.

4. Detailed description: Claude Generated the following using the above context:

Use case name	Solve Sudoku Puzzles from Files
Primary actor	User (runs the program from the command line)
Secondary actor	File System (provides puzzle1.txt through puzzle5.txt)
Goal	Display every solution of each of the five puzzles, or report that a puzzle has none.
Trigger	The user launches Assignment4.exe.
Preconditions	Assignment4.exe and the five puzzle files are in the working directory. Each file holds nine rows of nine entries, using digits 1-9 and '_' for blanks.
Main success flow	Steps 1-10 above.
Alternate flow: multiple solutions	If the search reaches a full grid more than once, each solution is printed and numbered (Solution 1, Solution 2, ...). The search continues until all possibilities are exhausted.
Alternate flow: no solution	If every candidate path dead-ends, the solution count stays at zero and the program prints "No solution found." (Report No Solution extends Find All Solutions.)
Exception flow: bad file	If a file is missing or has fewer than 81 readable entries, the program prints "Could not open or read file: <name>" and continues with the next file. (Report File Error extends Load Puzzle File.)
Included use cases	Load Puzzle File (read the grid from a file); Find All Solutions (recursive backtracking search); Display Results (print file name, puzzle, solutions).
Business rules	Each digit 1-9 appears exactly once in every row, column, and 3x3 block. Cells are filled by depth-first search with backtracking, and the program prints all solutions rather than stopping at the first.
Postconditions	All five files have been processed and each produced output. The program ends with exit code 0.

This diagram and description were created with the help of a GenAI (Claude, Anthropic) and reviewed against the program's actual behavior.